Documentation for URL Phishing Detection

What is this?

This is a webpage who detect a URL is Real or Not and it is also check the age of the domain so that we can find easily that it is Real or Not.

Adding these this things for improvements:

- Domain Age Check
- Google Safe Browsing API
- Screenshot Detection
- Deep Learning Model
- Real-time URL scanning

So our final idea after adding these things is this.

Why we need this?

Phishing is one of the **most common cyber attacks** today.

Example:

- Fake **bank website**
- Fake **Instagram / Facebook login page**
- Fake **UPI payment page**

These websites look real but they **steal user data**.

Example phishing URL:

```
https://paytm-login-security.xyz
```

Real URL:

```
https://paytm.com
```

So the goal of this project is:

Automatically detect whether a URL is safe or phishing.

This helps:

- users
- companies
- browsers
- security systems

Folder structure or files:

phishing-detector/

|

```
├── app.py
├── model/
│   └── phishing_model.pkl
│
├── utils/
│   ├── url_features.py
│   ├── domain_age.py
│   ├── safe_browsing.py
│   └── screenshot.py
│
├── static/
│   ├── style.css
│   └── script.js
│
├── templates/
│   └── index.html
│
├── dataset/
│   └── phishing_dataset.csv
│
├── train_model.py
└── requirements.txt
```

Tech Stack

Backend

- Python
- Flask

AI / ML

- Scikit-learn
- Pandas
- Numpy

Feature Extraction

- **tldextract**
- **whois**
- **requests**

Security APIs

- **Google Safe Browsing API**

Screenshot

- **Selenium**
- **Headless Chrome**

Frontend

- **HTML**
- **CSS**
- **JavaScript**

Steps to build this project:

Step 1 — Create Project Folder

First create your main project directory.

```
phishing-detector/
```

Inside it create folders:

```
phishing-detector
│
├── dataset
├── model
├── backend
├── frontend
├── utils
└── app.py
```

Step 2 — Get Dataset (Very First Task)

Before coding anything, you need **training data**.

Download phishing dataset from:

- Kaggle
- UCI ML Repository

Example dataset columns:

```
URL
URL_Length
Has_IP
Num_Dots
HTTPS
Domain_Age
Label
```

Where:

```
0 = Legitimate
1 = Phishing
```

Save it inside:

```
dataset/phishing_dataset.csv
```

Step 3 — Train the Machine Learning Model

Now build the **ML detection model**.

Create file:

```
train_model.py
```

Workflow:

```
Dataset
   ↓
Feature Extraction
   ↓
Train ML Model
   ↓
Save Model (.pkl)
```

Model options:

- Random Forest ⭐ (recommended)
- XGBoost
- Logistic Regression

Output:

```
model/phishing_model.pkl
```

This file is your **AI brain**.

Step 4 — Build URL Feature Extractor

Create a file:

```
utils/url_features.py
```

This script will extract features from user URL.

Example features:

```
URL length
number of dots
number of special characters
HTTPS present or not
IP address in URL
```

Example:

Input:

```
https://secure-paypal-login.xyz
```

Output features:

```
length = 28
dots = 2
special characters = 3
https = 1
ip address = 0
```

These features go into the ML model.

Step 5 — Domain Age Checker

Create file:

```
utils/domain_age.py
```

Use **WHOIS API**.

Logic:

```
Get domain creation date
Current date - creation date
```

Example:

```
Domain: example.com
Age: 12 years
```

Phishing sites usually:

```
Age < 6 months
```

Step 6 — Google Safe Browsing API

Create:

```
utils/safe_browsing.py
```

Use Google API to check if URL is:

```
phishing
malware
dangerous
```

Output example:

```
Status: Blacklisted
```

Step 7 — Screenshot Detection (Optional Advanced)

Create:

```
utils/screenshot.py
```

Use:

```
Selenium
Headless Chrome
```

Take screenshot of website.

Later you can use **Deep Learning CNN** to detect:

```
Fake login pages
Fake banking UI
```

This makes your project **very advanced**.

Step 8 — Build Backend (Main Logic)

Now create backend using **Flask**.

File:

```
app.py
```

Responsibilities:

```
Receive URL from user
↓
Extract features
↓
Check domain age
↓
Check Google Safe Browsing
↓
Run ML model
↓
Return result
```

Example response:

```
{
 "url": "example.com",
 "prediction": "Phishing",
 "risk_score": 87
}
```

Step 9 — Build Frontend (Now UI)

Now build the website.

Create folder:

```
frontend
```

Structure:

```
frontend
│
├── index.html
├── style.css
└── script.js
```

index.html

Input box for URL.

Example UI:

```
Enter URL: [_______________]

        [ Analyze ]
```

style.css

Design page:

```
centered input
cybersecurity theme
dark background
```

script.js

This file sends URL to backend.

Flow:

```
User enters URL
↓
JS sends request to Flask API
↓
Backend processes
↓
Result displayed
```

Step 10 — Connect Frontend + Backend

Use **Fetch API**

Example flow:

```
HTML button click
     ↓
JavaScript fetch()
     ↓
Flask API /analyze
     ↓
Return result
     ↓
Display result
```

Result shown like:

```
Result: Phishing ✕
Confidence: 91%
```

Final System Architecture

```
User
  ↓
Frontend (HTML CSS JS)
  ↓
Flask Backend
  ↓
Feature Extraction
  ↓
Domain Age Check
  ↓
Google Safe Browsing
  ↓
ML Model
  ↓
Result
```

Final Folder Structure (Clean Version)

```
phishing-detector
│
├── dataset
│   └── phishing_dataset.csv
│
├── model
│   └── phishing_model.pkl
│
├── utils
│   ├── url_features.py
│   ├── domain_age.py
│   ├── safe_browsing.py
│   └── screenshot.py
│
├── frontend
│   ├── index.html
│   ├── style.css
│   └── script.js
│
├── train_model.py
├── app.py
└── requirements.txt
```

Time Required

If you follow this plan:

```
Dataset + ML model → 1 day
Backend → 1 day
Frontend → 4 hours
API integrations → 1 day
Testing → 4 hours
```

Total:

```
3-4 days project
```

USP:

• Machine Learning detection

• Domain age analysis

• Google Safe Browsing integration

• Screenshot based phishing detection

• Real-time URL scanning